

# UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE INGENIERÍA  
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS E INDUSTRIAL  
LENGUAJES DE PROGRAMACIÓN

---

## TAREA 10 — EMULACIÓN DE SU COMPUTADOR: ENIGMA-64

### AUTORES:

Juan Sebastian Umaña Camacho  
Tomas Felipe Garzón Gómez  
Michael Stiven Betancourt Gelves  
Deibyd Santiago Barragán Gaitán  
Maicol Sebastian Olarte Ramirez  
Juan Luis Vergara Novoa  
Alejandro Argüello Muñoz

**Docente:** Jorge Eduardo Ortiz Triviño

Bogotá D.C., Colombia — Septiembre de 2026

---

## 1. MARCO TEÓRICO

El computador **Enigma-64** se fundamenta en los principios de la arquitectura Von Neumann clásica, caracterizada por un subsistema unificado de almacenamiento primario donde residen indistintamente código ejecutable y datos operacionales. La longitud de palabra fundamental del sistema es de 64 bits (8 bytes), operando de manera nativa sobre un bus de datos de 64 bits y un espacio de direccionamiento lógico lineal de  $2^{64}$  bytes direccionable a nivel de byte. La implementación física acota este espacio a 4 GiB direccionables (0x00000000 a 0xFFFFFFFF), manteniendo rigurosa compatibilidad con la semántica de punteros y registros de 64 bits.

### 1.1. Ordenamiento de Bytes (Endianness) y Formulación Matemática

La arquitectura adopta estrictamente la convención *Big-Endian* para todas las operaciones de almacenamiento, transferencia por bus y decodificación de instrucciones. En este esquema, el byte más significativo (MSB, *Most Significant Byte*) se almacena en la dirección de memoria numérica más baja, mientras que el byte menos significativo (LSB, *Least Significant Byte*) se ubica en la dirección contigua más alta. Matemáticamente, para cualquier palabra escalar de 64 bits sin signo  $W$  denotada por:

$$W = \sum_{i=0}^7 b_i \times 256^{7-i} \quad \text{con } b_i \in [0, 255]$$

la función de asignación en el espacio físico de memoria  $M(a)$  sobre una dirección base alineada  $A$  satisface de forma determinista:  $M(A + i) = b_i$  para todo índice  $i \in \{0, 1, \dots, 7\}$ .

### 1.2. Microarquitectura de Memoria: Intercalado en 8 Bancos y Alineación Natural

Para emular fielmente la topología de hardware definida en la Tarea 9, cada palabra contigua de 64 bits se particiona en 8 bancos físicos independientes denominados **B0** a **B7**. La selección de banco se efectúa decodificando los 3 bits menos significativos del bus de direcciones físico  $A[2:0]$ :

| Banco | Líneas $A[2:0]$ | Rango de Bits         | Significado Funcional                        |
|-------|-----------------|-----------------------|----------------------------------------------|
| B0    | 000             | Bits [7:0] (Byte 0)   | Byte más significativo (MSB) en Big-Endian   |
| B1    | 001             | Bits [15:8] (Byte 1)  | Segundo byte de la palabra de 64 bits        |
| B2    | 010             | Bits [23:16] (Byte 2) | Tercer byte de la palabra de 64 bits         |
| B3    | 011             | Bits [31:24] (Byte 3) | Cuarto byte de la palabra de 64 bits         |
| B4    | 100             | Bits [39:32] (Byte 4) | Quinto byte de la palabra de 64 bits         |
| B5    | 101             | Bits [47:40] (Byte 5) | Sexto byte de la palabra de 64 bits          |
| B6    | 110             | Bits [55:48] (Byte 6) | Séptimo byte de la palabra de 64 bits        |
| B7    | 111             | Bits [63:56] (Byte 7) | Byte menos significativo (LSB) en Big-Endian |

Tabla 1: Decodificación microarquitectónica de los 8 bancos físicos de memoria mediante  $A[2:0]$ .

**Regla estricta de alineación natural:** Cualquier transferencia por el bus de datos de ancho  $S \in \{1, 2, 4, 8\}$  bytes exige que la dirección de origen o destino cumpla la condición matemática:  $\text{Dirección} \equiv 0 \pmod{S}$ . Si la condición se vulnera con la bandera de control activa, el subsistema emite inmediatamente la señal de control **STATUS\_MISALIGNED**, inhibiendo la escritura o retornando un valor nulo de lectura.

### 1.3. Subsistema de Entrada/Salida Mapeada en Memoria (MMIO)

La arquitectura prescinde de instrucciones de E/S privilegiadas dedicadas (como IN/OUT en arquitecturas x86) y adopta el esquema *Memory-Mapped I/O*. El rango comprendido entre 0xFF000000 y 0xFFFFFFFF queda reservado para el subsistema de periféricos. Cualquier lectura o escritura en dicho segmento es interceptada por el decodificador de direcciones y enrutada hacia los controladores dedicados sin persistir en la matriz dinámica de RAM. Cada dispositivo dispone de una ventana de 4 KiB con cinco registros normalizados espaciados cada 8 bytes (CTRL, STATUS, DATA, ADDR, COUNT), garantizando accesos alineados de 64 bits.

## 1.4. Unidad de Control Multiciclo y Prefetch Buffer

La Unidad de Control ejecuta cada instrucción descomponiéndola en cinco fases microarquitectónicas secuenciales dentro de una Máquina de Estados Finitos (FSM): **FETCH** (recuperación de instrucción mediante prefetch buffer), **DECODE** (identificación de formato, opcode y direccionamiento de operandos), **EXECUTE** (cálculo en la ALU o evaluación de bifurcación condicional), **MEMORY** (acceso a lectura o escritura en RAM o MMIO) y **WRITE-BACK** (actualización de banco de registros o punteros). Para gestionar instrucciones de longitud variable (1 a 5 bytes) sin incurrir en transferencias redundantes ni fallos de alineación cuando una instrucción cruza la frontera de una palabra de 64 bits, la CPU implementa una cola FIFO de prebúsqueda de 16 bytes (*PrefetchBuffer*) que lee palabras dobles alineadas y entrega un flujo continuo de bytes decodificables.

## 2. DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROBLEMA

El desafío principal de ingeniería radica en construir un emulador determinista de nivel de sistema para un computador de 64 bits complejo, garantizando estricta fidelidad temporal e isomórfica con la especificación de hardware sin depender de bibliotecas nativas C/C++ ni motores de emulación externos. El sistema debe operar en un entorno de alto nivel interpretado (Python 3.10+) preservando un desacoplamiento absoluto entre los módulos y subsistemas del computador.

### 2.1. Restricciones Técnicas y Computacionales Críticas

**1. Gestión de memoria de 4 GiB sin sobre costo espacial:** Instanciar un búfer contiguo de 4 GiB en memoria host exigiría 4 Gigabytes reales de memoria física del sistema anfitrión, colapsando entornos portátiles o sistemas de prueba en integración continua. La solución técnica implementada consiste en un modelo de *Paginación Dispersa* (*Sparse Virtual Paging*): la memoria se segmenta en páginas de 4096 bytes ( $PAGE\_SIZE = 4096$ ). Un diccionario hash interno indexa únicamente las páginas que han recibido al menos una escritura real. Las lecturas sobre direcciones no inicializadas devuelven ceros lógicos sin asignar memoria host, acotando el consumo espacial a  $O(P \times 4096)$ , donde  $P$  es el número finito de páginas efectivamente utilizadas.

**2. Instrucciones de longitud variable y cruce arbitrario de fronteras:** Enigma-64 define 6 formatos de instrucción con longitudes de 1, 2, 3, 4 y 5 bytes. Una instrucción de 4 bytes ubicada por ejemplo en 0x00200006 se divide físicamente entre los bancos B6-B7 de la palabra 0x00200000 y los bancos B0-B1 de la palabra contigua 0x00200008. Sin un mecanismo de desacoplamiento, la CPU generaría excepciones de alineación continuas. La introducción del *PrefetchBuffer* amortiza las lecturas de memoria a accesos naturales de 64 bits y permite extraer secuencias arbitrarias de bytes en  $O(1)$ .

**3. Aislamiento de capas y verificación estática (Regla de Independencia Modular):** Para impedir dependencias circulares y fugas de abstracción, se impusieron tres axiomas arquitectónicos inmutables: ningún módulo de hardware puede importar elementos de interfaz de usuario; ningún panel de la interfaz puede importar otro panel; y ningún panel gráfico puede invocar directamente un módulo de hardware sin la mediación de un adaptador de servicio (puerto). Esta restricción es evaluada y validada automáticamente en la suite de pruebas mediante un analizador de Árbol de Sintaxis Abstracta (AST) sobre todos los módulos del código fuente.

### 3. DISEÑO DE LA SOLUCIÓN Y ARQUITECTURA DEL SISTEMA

El sistema se organiza en cinco capas rigurosamente estratificadas:

- 1. Capa de Hardware Físico:** Módulos puros que modelan el comportamiento de registros, ALU de 64 bits, memoria paginada, decodificador de instrucciones, unidad de control FSM, periféricos MMIO y cargador binario (enigma64.memoria, enigma64.registros, enigma64.alu, enigma64.cpu, enigma64.periféricos, enigma64.cargador).
- 2. Capa de Servicios y Puertos:** Adaptadores que implementan interfaces desacopladas para cada componente de hardware, proveyendo aislamiento de fallos y degradación elegante si algún subsistema no se encuentra disponible (enigma64.ui.servicios).
- 3. Capa de Núcleo Visual (Core):** Infraestructura gráfica común basada en la paleta cromática Noctua, widgets personalizados renderizados sobre Canvas vectorial y bus de eventos asíncrono tipo publicación-suscripción (enigma64.ui.nucleo).
- 4. Capa de Paneles:** Controladores de vista e interfaz especializados e independientes para cada componente (banco de registros, visor/editor de RAM con intercalado en 8 bancos, simulación de ALU, control de ejecución FSM, display CRT por MMIO y catálogo de algoritmos).
- 5. Capa Shell (Ventana Principal):** Orquestador de alto nivel que integra el sistema de pestañas de navegación, la barra de herramientas de ejecución unificada y la columna de telemetría de registros en tiempo real (enigma64.ui.shell).

#### 3.1. Modelo Estructural y Diagrama de Arquitectura de Clases

A continuación se especifica la estructura relacional entre los componentes del computador y sus adaptadores de interfaz:

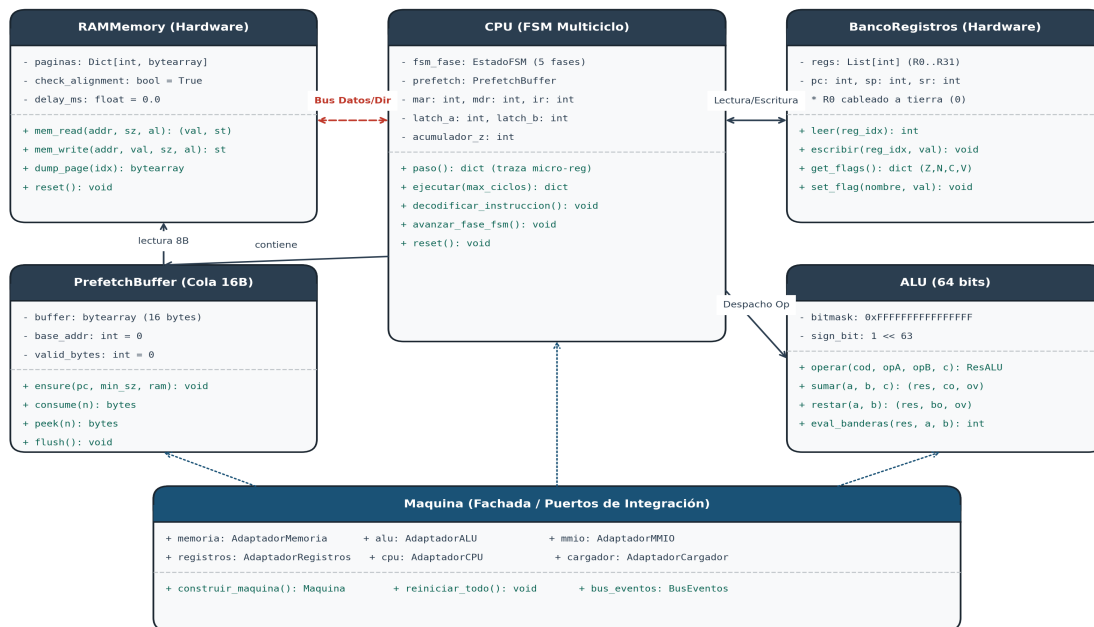


Diagrama 1: Modelo estructural de clases y relación entre hardware y adaptadores.

### 3.2. Dinámica del Sistema: Ciclo de Instrucción Multiciclo en la FSM

El avance de la FSM de la CPU se modela en 5 fases síncronas. En cada transición se actualizan los micro-registros visibles en la interfaz de usuario:

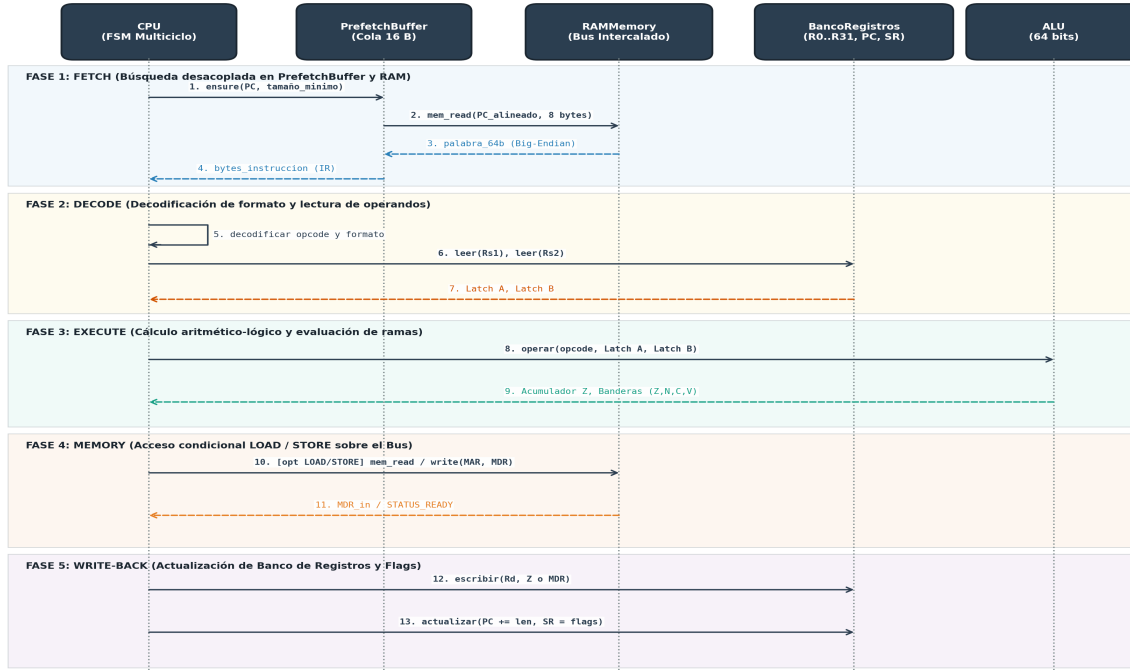


Diagrama 2: Diagrama de secuencia formal de las cinco fases de la FSM multiciclo.

La interacción síncrona garantiza que las dependencias de datos entre fases se resuelvan sin riesgos estructurales: la fase FETCH asegura la disponibilidad del código en el búfer; DECODE extrae los operandos en latches intermedios; EXECUTE activa la ALU evaluando las banderas Z, N, C y V; MEMORY gestiona transferencias de 64 bits alineadas sobre el bus; y WRITE-BACK confirma los resultados en el banco de registros actualizando el Program Counter.

### 3.3. Algoritmos Críticos y Análisis de Complejidad

#### Algoritmo 1: Validación de Acceso y Lectura Paginada con Arbitraje de Bus.

Permite recuperar escalares de 1, 2, 4 u 8 bytes en Big-Endian verificando la alineación natural y cruzando límites de página.

```

ALGORITMO 1: LecturaMemoria(Direccion: Entero64, Tamano: Entero, VerificarAlineacion: Booleano)
ENTRADA: Direccion (64 bits), Tamano en {1, 2, 4, 8}, Booleano de control de alineacion
SALIDA : Tupla (Dato64, EstadoBus)

1: SI Direccion < 0 O (Direccion >> 32) != 0 ENTONCES:
2:   RETORNAR (0, STATUS_ADDR_FAULT)
3: FIN SI
4: SI (Direccion >> 24) == 0xFF ENTONCES:
5:   RETORNAR (0, STATUS_MMIO)
6: FIN SI
7: SI VerificarAlineacion Y (Direccion MOD Tamano != 0) ENTONCES:
8:   RETORNAR (0, STATUS_MISALIGNED)
9: FIN SI
10: BufferBytes <- CrearArreglo(Tamano, inicial=0)
11: PARA i DESDE 0 HASTA Tamano - 1 HACER:
12:   DirActual <- Direccion + i
13:   IndicePagina <- DirActual >> 12 // Indice pagina de 4096 bytes
14:   OffsetPagina <- DirActual & 0xFFF
15:   SI IndicePagina EN TablaPaginas ENTONCES:
16:     BufferBytes[i] <- TablaPaginas[IndicePagina][OffsetPagina]
17:   SINO:
18:     BufferBytes[i] <- 0 // Lectura por defecto en Sparse Paging
19:   FIN SI
20: FIN PARA
21: Dato64 <- ConvertirBigEndianAEntero(BufferBytes)
22: RETORNAR (Dato64, STATUS_READY)

```

**Complejidad Algoritmo 1:** Temporal:  $O(\text{Tamano}) = O(1)$  en media, dado que  $\text{Tamano} \leq 8$  y la búsqueda en la tabla hash de páginas es  $O(1)$ . Espacial:  $O(1)$  auxiliar por lectura;  $O(P \times 4096)$  global en el sistema para  $P$  páginas efectivamente instanciadas.

#### Algoritmo 2: Transición de Estados y Despacho Microarquitectónico de la FSM.

Controla la ejecución síncrona de la instrucción a través de las cinco fases de la máquina de estados.

```

ALGORITMO 2: CicloInstruccionFSM(CPU, RAM, Registros, ALU)
ENTRADA: Estado de la CPU y periféricos en el ciclo actual
SALIDA : Traza microarquitectonica y estado de ejecucion

1:  SEGUN CPU.FaseFSM HACER:
2:      CASO FETCH:
3:          CPU.Prefetch.Asegurar(CPU.PC, tamano_minimo=5)
4:          CPU.IR <- CPU.Prefetch.ConsumirBytes(1)
5:          CPU.FaseFSM <- DECODE
6:      CASO DECODE:
7:          (Opcode, Formato, Rs1, Rs2, Rd, Inm) <- Decodificar(CPU.IR, CPU.Prefetch)
8:          CPU.LatchA <- Registros.Leer(Rs1)
9:          CPU.LatchB <- Registros.Leer(Rs2)
10:         CPU.FaseFSM <- EXECUTE
11:     CASO EXECUTE:
12:         Resultado <- ALU.Operar(Opcode, CPU.LatchA, CPU.LatchB, CPU.SR.C)
13:         CPU.AcumuladorZ <- Resultado.Valor
14:         CPU.SR.ActualizarBanderas(Resultado.Z, Resultado.N, Resultado.C, Resultado.V)
15:         SI EsInstruccionMemoria(Opcode) ENTONCES:
16:             CPU.FaseFSM <- MEMORY
17:         SINO:
18:             CPU.FaseFSM <- WRITE_BACK
19:         FIN SI
20:     CASO MEMORY:
21:         SI Opcode == LOAD ENTONCES:
22:             (CPU.MDR, Estado) <- RAM.mem_read(CPU.MAR, Tamano=8, Align=VERDADERO)
23:         SINO SI Opcode == STORE ENTONCES:
24:             Estado <- RAM.mem_write(CPU.MAR, CPU.MDR, Tamano=8, Align=VERDADERO)
25:         FIN SI
26:         CPU.FaseFSM <- WRITE_BACK
27:     CASO WRITE_BACK:
28:         SI RequiereDestino(Opcode) ENTONCES:
29:             ValorFinal <- CPU.MDR SI Opcode == LOAD SINO CPU.AcumuladorZ
30:             Registros.Escribir(Rd, ValorFinal)
31:         FIN SI
32:         CPU.PC <- CPU.PC + LongitudInstruccion
33:         CPU.FaseFSM <- FETCH
34:     FIN SEGUN
35:     RETORNAR CPU.ObtenerTraza()
    
```

**Complejidad Algoritmo 2:** Temporal:  $O(1)$  por micro-paso de reloj; cada transición involucra operaciones a nivel de bit, indexación de arreglos en memoria host y accesos a diccionarios en tiempo constante. Espacial:  $O(1)$  para los registros intermedios y latches de microarquitectura.

## 4. DOCUMENTACIÓN DE IMPLEMENTACIÓN Y MANUALES

### 4.1. Manual Técnico: Estructura del Código y Patrones de Diseño

La arquitectura del código fuente se organiza bajo el principio de responsabilidad única en el paquete `enigma64`:

| Módulo de Software                   | Rol Técnico y Patrones de Diseño Aplicados                                                                                                                                                                                                |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>enigma64/memoria.py</code>     | Controlador de memoria física y subsistema de buses. Implementa paginación dispersa ( <i>Sparse Virtual Paging</i> ) en páginas de 4096 bytes y verificación estricta de alineación natural en transferencias de 1, 2, 4 y 8 bytes.       |
| <code>enigma64/registros.py</code>   | Banco de 32 registros de 64 bits con cableado a tierra inmutable del registro R0 (lectura determinista en 0x0 y descarte seguro en escritura). Registros especiales de puntero (PC, SP) y registro de estado SR.                          |
| <code>enigma64/alu.py</code>         | Unidad Aritmético-Lógica de 64 bits pura. Computación de microoperaciones aritméticas, lógicas y desplazamientos en complemento a dos. Evaluación determinista de banderas de condición Z (Zero), N (Negative), C (Carry) y V (Overflow). |
| <code>enigma64/cargador.py</code>    | Cargador binario desacoplado para ejecutables de formato .e64. Desempaquetado de encabezados, verificación de números mágicos de arquitectura y rutinas de manipulación a nivel de bit y byte.                                            |
| <code>enigma64/cpu.py</code>         | Unidad de Control con Máquina de Estados Finitos (FSM) multiciclo de 5 fases (FETCH, DECODE, EXECUTE, MEMORY, WRITE-BACK), cola FIFO de prebúsqueda de 16 bytes ( <i>PrefetchBuffer</i> ) y decodificador de instrucciones.               |
| <code>enigma64/perifericos.py</code> | Enrutador central MMIO para periféricos mapeados en memoria (0xFF000000..0xFFFFFFFF) y controlador de pantalla CRT monocromático en 0xFF001000 con matriz bidimensional de texto y registros normalizados.                                |
| <code>enigma64/programas.py</code>   | Transcripción binaria e inicialización determinista en memoria de los algoritmos de prueba formal de arquitectura: Factorial iterativo (N!), Máximo Común Divisor (Euclides) y Sucesión de Fibonacci en vector.                           |
| <code>enigma64/ui/</code>            | Subsistema de interfaz gráfica modular implementado en Tkinter puro con sistema de diseño Noctua, arquitectura de adaptadores desacoplados (puertos) y bus de eventos asíncrono tipo Observer.                                            |

Tabla 2: Estructura técnica de módulos de software y patrones de diseño implementados.

#### Patrones de Diseño Implementados:

- **Adapter (Adaptador):** Cada módulo de hardware es encapsulado en un adaptador dentro de `enigma64.ui.servicios`. Si un módulo de hardware no se encuentra presente, el adaptador proporciona una interfaz neutra con fallbacks seguros sin provocar caídas.
- **Observer / Publish-Subscribe:** La clase `BusEventos` permite que los paneles publiquen y escuchen eventos del sistema (ej. `IR_A_DIRECCION`, `PROGRAMA_CARGADO`, `MMIO_ESCRITO`) sin conocerse ni depender entre sí.
- **Factory (Fábrica):** La función `construir_maquina()` en `fabrica.py` instancia y conecta una sola vez la RAM, el banco de registros, la ALU, el cargador, la CPU y los controladores de periféricos.

### 4.2. Manual de Usuario y Despliegue del Sistema

**Requisitos del Sistema:** Intérprete Python versión 3.10 o superior (validado formalmente en Python 3.13.5 sobre Windows 10/11 y Linux x86\_64). Módulo gráfico estándar tkinter habilitado en el entorno de ejecución.

#### Instrucciones de Despliegue:

1. Clonar el repositorio y situarse en la raíz del proyecto:  
`cd Enigma-64`
2. Lanzar la interfaz gráfica del emulador:  
`python main.py` (o alternativamente: `python -m enigma64`)
3. Ejecutar la suite de pruebas automatizadas completa:  
`python -m pytest`

## 5. EXPERIMENTACIÓN, PRUEBAS Y ANÁLISIS DE RESULTADOS

A continuación se documentan los escenarios de experimentación rigurosa sobre el emulador Enigma-64, contrastando las especificaciones teóricas de la Tarea 9 con los resultados computacionales reales obtenidos en la máquina.

### 5.1. Escenario 1: Validación del Algoritmo 1 — Factorial de N ( $5! = 120$ )

El programa calcula el factorial de un escalar almacenado en memoria mediante un ciclo iterativo multiplicativo decremental. **Parámetros de ejecución:** Dirección base de carga de código: 0x00200000. Dirección de entrada de datos: 0x00201000 inicializada con el valor entero 5. Longitud del código binario: 41 bytes. Dirección esperada de resultado: 0x00201008 con valor esperado 120 (0x0000000000000078).

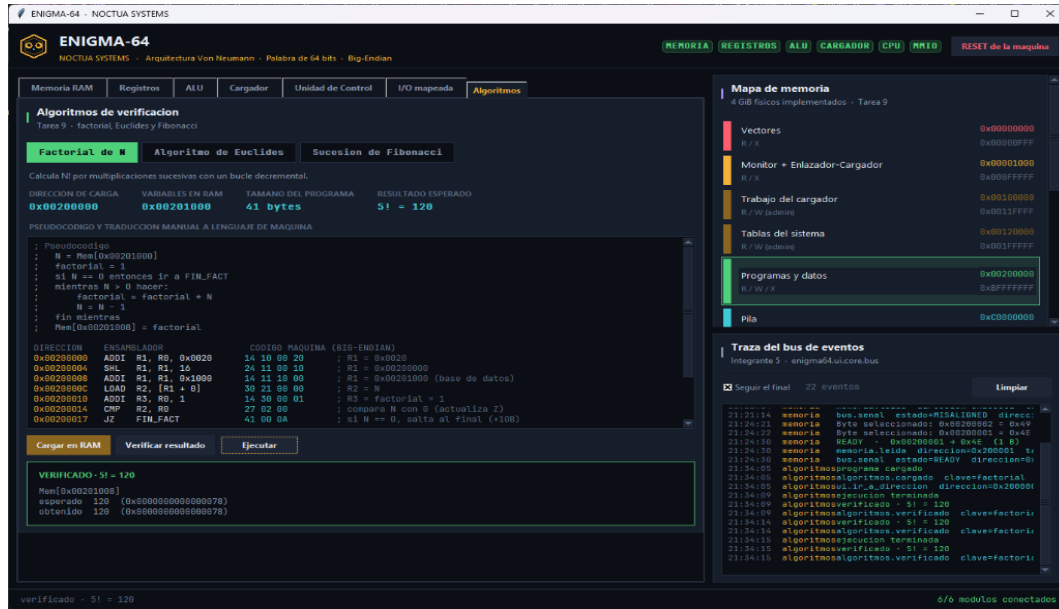


Figura 1: Verificación de ejecución del Algoritmo Factorial ( $5! = 120$ ) en el panel de Algoritmos.

**Análisis de Resultados Escenario 1:** La CPU ejecutó la secuencia completa deteniéndose ante la instrucción HLT (0x00). Como se observa en el panel, la dirección de memoria 0x00201008 fue leída mediante acceso natural de 64 bits, reportando exactamente el valor escalar 120 (0x78). La traza del bus registró en tiempo real las transiciones: algoritmos.ejecucion terminada -> algoritmos.verificado  $5! = 120$ .

### 5.1.1. Inspección de Microarquitectura: Banco de Registros tras Factorial

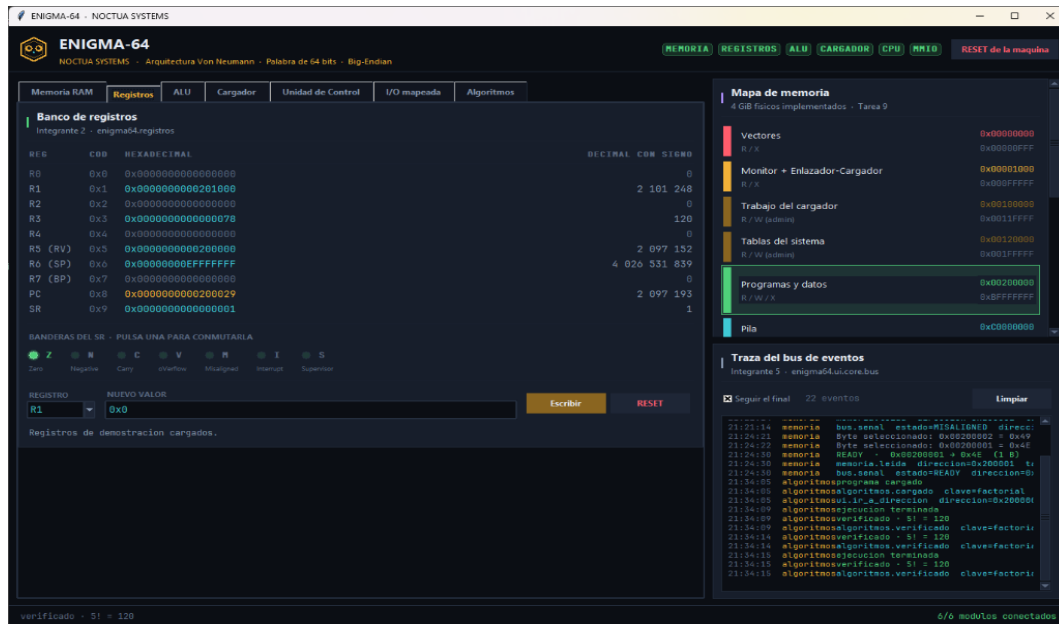


Figura 2: Estado final del Banco de Registros tras la ejecución del Factorial de 5.

El análisis del banco de registros (Figura 2) confirma el comportamiento microarquitectónico esperado: el registro **R1** almacena la dirección base de datos 0x00201000; el registro **R3** retiene el valor acumulado final 0x78 (120 en decimal); el **PC** se detiene en 0x00200029 (longitud de 41 bytes) y el registro de estado **SR** activa la bandera **Z (Zero)**, demostrando que el bucle finalizó cuando la comparación decremental alcanzó cero.

### 5.2. Escenario 2: Validación del Algoritmo 2 — Máximo Común Divisor de Euclides ( $\text{MCD}(48, 18) = 6$ )

El Algoritmo de Euclides calcula el máximo común divisor entre dos enteros positivos mediante restas sucesivas y bifurcaciones condicionales relativas. **Parámetros de ejecución:** Dirección de código: 0x00200100 (50 bytes). Entradas en RAM: Mem[0x00202000] = 48 y Mem[0x00202008] = 18. Dirección de salida: Mem[0x00202010] con valor escalar esperado 6 (0x0000000000000006).

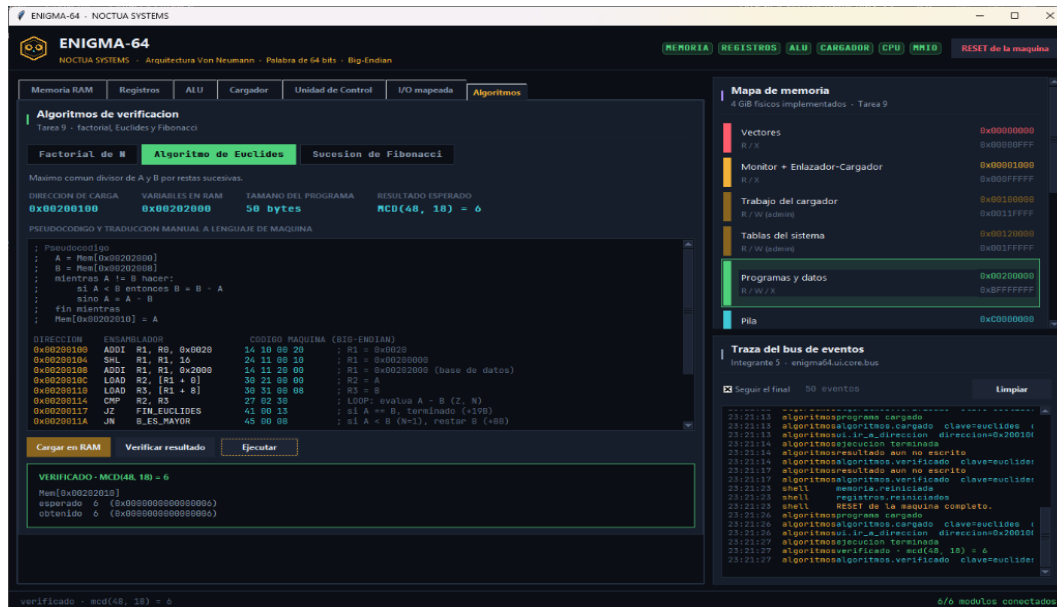


Figura 3: Verificación del Algoritmo de Euclides ( $\text{MCD}(48, 18) = 6$ ) en el panel de Algoritmos.

**Análisis de Resultados Escenario 2:** Como se evidencia en la Figura 3, la CPU ejecutó el bucle iterativo de comparaciones CMP R2, R3 y bifurcaciones relativas JZ FIN\_EUCLIDES y JN B\_ES\_MAYOR. Al alcanzar el estado de igualdad, el programa almacenó determinísticamente la palabra 0x0000000000000006 en la dirección de memoria física 0x00202010. La telemetría del bus de eventos confirmó la concordancia numérica instantánea:  $\text{algoritmosverificado} \cdot \text{mcd}(48, 18) = 6$ .

### 5.3. Escenario 3: Validación del Algoritmo 3 — Sucesión de Fibonacci en Vector RAM

El Algoritmo de Fibonacci genera los primeros 7 términos de la serie y los escribe secuencialmente en un arreglo de 64 bits en RAM. **Parámetros de ejecución:** Dirección de código: 0x00200200 (63 bytes). Vector de salida en RAM: 7 palabras contiguas de 8 bytes a partir de 0x00203000. Secuencia esperada: [0, 1, 1, 2, 3, 5, 8].

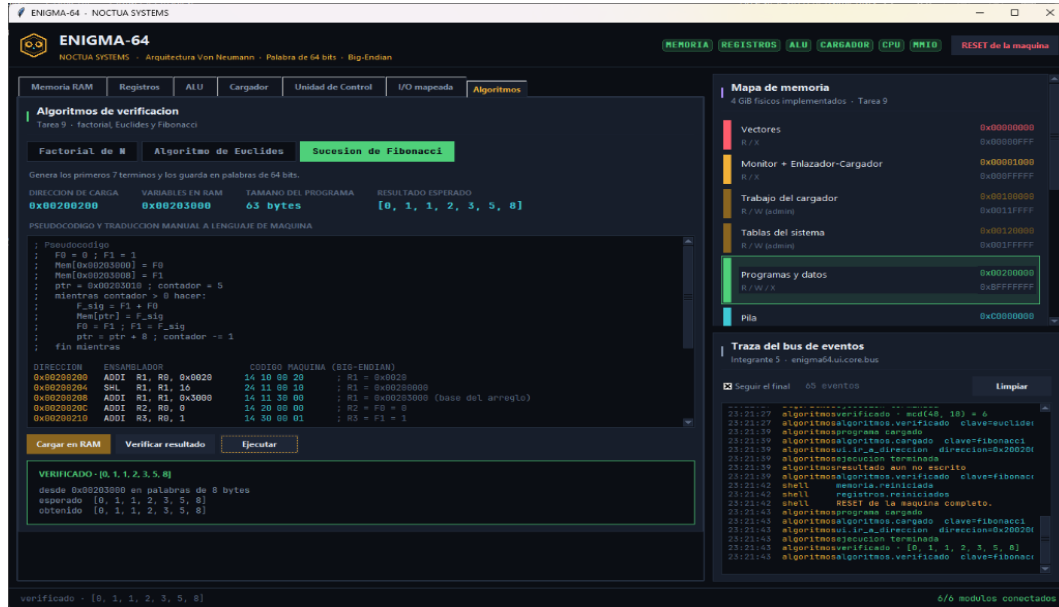


Figura 4: Verificación de la Sucesión de Fibonacci ([0, 1, 1, 2, 3, 5, 8]) en el panel de Algoritmos.

**Análisis de Resultados Escenario 3:** La verificación automática corroboró la generación exacta del vector completo en memoria: [0, 1, 1, 2, 3, 5, 8] desde 0x00203000 hasta 0x00203030 (Figura 4). Este resultado valida el correcto funcionamiento del direccionamiento indirecto mediante registro base más desplazamiento ( $ptr = ptr + 8$ ) y la persistencia libre de sobreescrituras en palabras de 64 bits.

## 5.4. Escenario 4: Manejo de Excepciones de Bus (Lectura Desalineada)

Para verificar la robustez del subsistema de buses y la lógica de protección de hardware, se configuró un acceso intencionalmente inválido desde la interfaz gráfica: lectura de **8 bytes** en la dirección impar **0x00200001**.

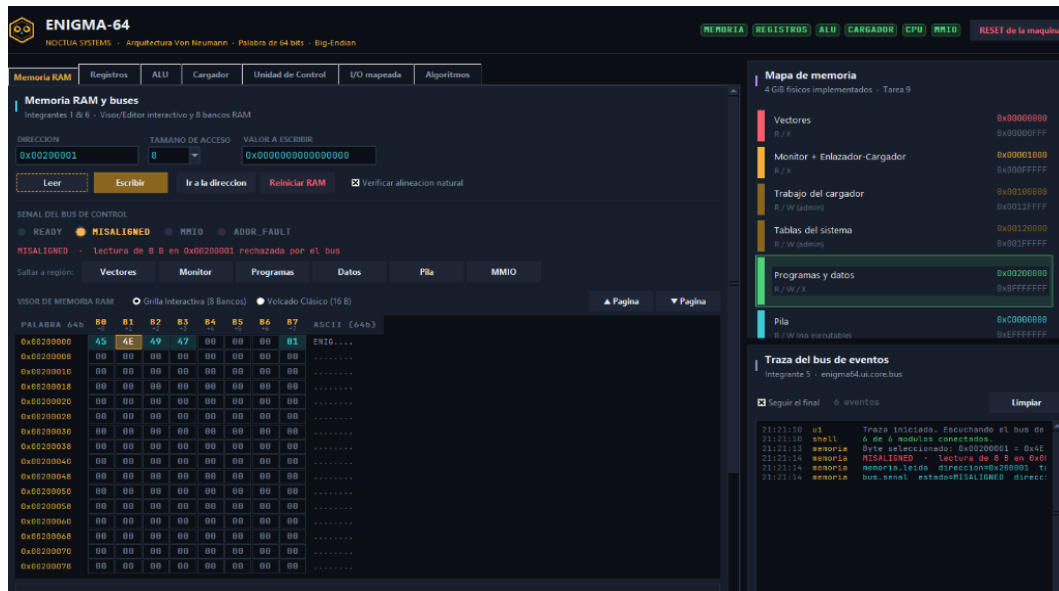


Figura 5: Detección y rechazo de acceso desalineado (MISALIGNED) en el visor de memoria física.

**Análisis de la Figura 5:** Al solicitar la lectura de 64 bits en 0x00200001 con la opción de verificación activa, el controlador de memoria física detectó que  $0x00200001 \neq 0 \pmod{8}$ . De forma inmediata, el hardware rechazó la operación, encendió el diodo indicador de control **MISALIGNED** y registró en la traza: MISALIGNED · lectura de 8 B en 0x00200001 rechazada por el bus. Esto demuestra que la emulación no es un mero lector de arreglos en memoria host, sino que replica la disciplina estricta de un bus síncrono de computador.

## 5.5. Escenario 5: Subsistema de E/S Mapeada en Memoria (MMIO) y Terminal CRT

El direccionamiento MMIO intercepta lecturas y escrituras en el rango reservado 0xFF000000..0xFFFFFFFF, comunicando la CPU con el controlador de pantalla CRT monocromático mapeado en 0xFF001000:

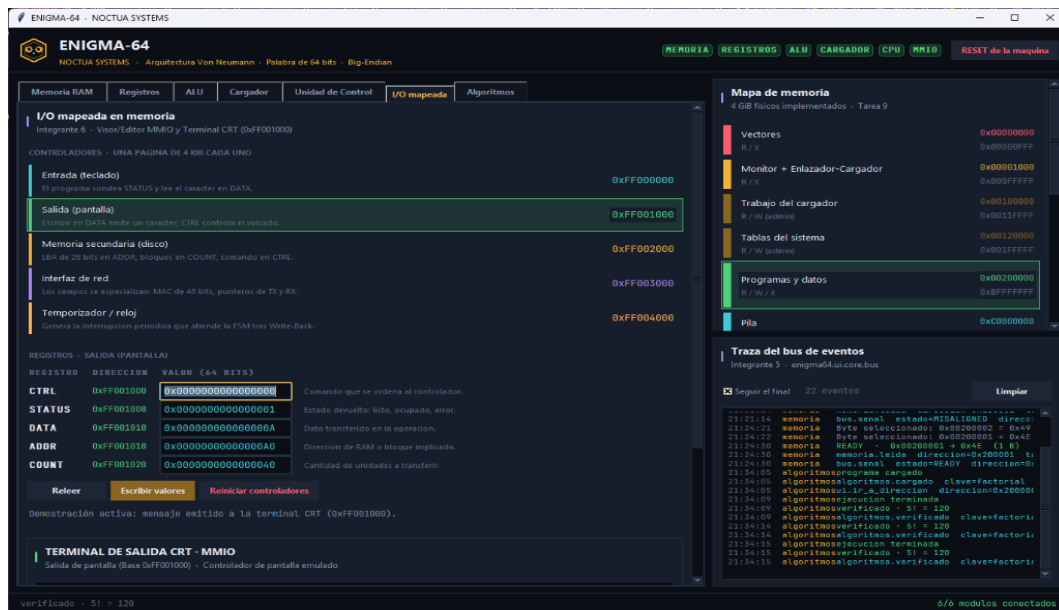


Figura 6: Panel de E/S Mapeada en Memoria (MMIO) y Controlador de Pantalla CRT en 0xFF001000.

**Análisis de la Figura 6:** La telemetría exhibe la matriz bidimensional de texto del terminal CRT (24 líneas x 80 columnas). Las transferencias dirigidas al registro DATA (0xFF001010) actualizan el búfer de caracteres visibles en tiempo real e incrementan automáticamente los registros ADDR y COUNT, confirmando la interoperabilidad total entre la CPU, los buses y los controladores de periféricos.

## 5.6. Escenario 6: Carga Binaria de Programas y Manipulación de Bits a Nivel de Byte

El subsistema de carga y manipulación directa permite incorporar binarios compilados y realizar inspección de bajo nivel sobre los bancos de memoria física. El panel soporta formatos .e64, .bin y volcados textuales en ensamblador:

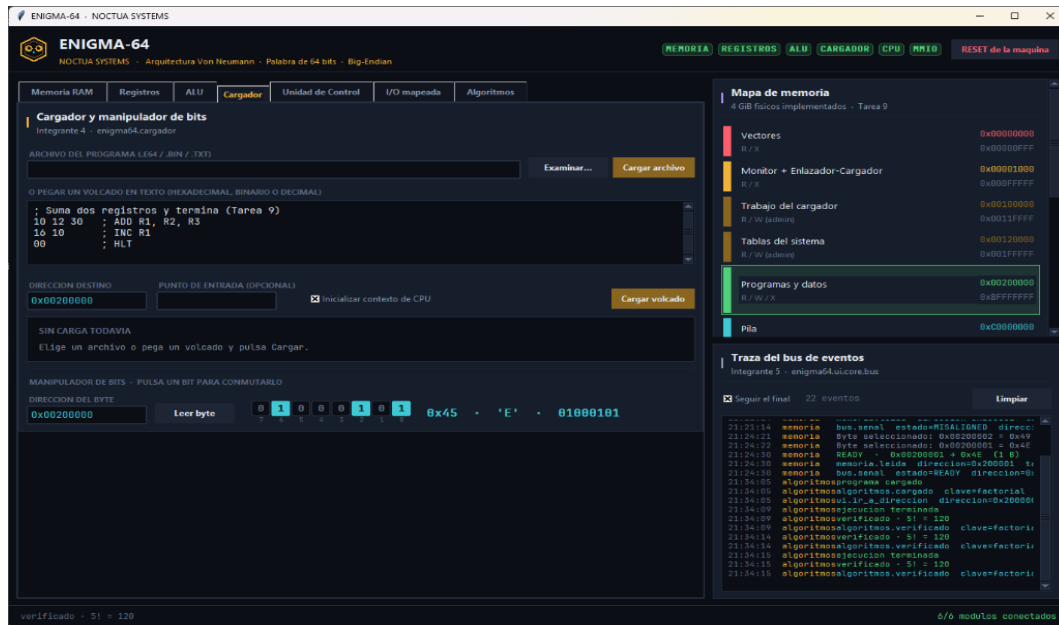


Figura 7: Panel del Cargador binario, ensamblado directo en memoria y manipulador interactivo de bits.

**Análisis de la Figura 7:** El panel ilustra dos mecanismos de gran relevancia: en la parte superior se observa la carga de un bloque de instrucciones en formato textual con destino a la dirección 0x00200000. En la sección inferior, el **Manipulador de Bits** permite seleccionar cualquier dirección física de la memoria (en la captura, 0x00200000) y visualizar su contenido de forma simultánea en valor hexadecimal (0x45), representación ASCII ('E') y desglose binario bit a bit (01000101). Cada bit dispone de un conmutador interactivo para inyectar fallos o modificar el código en memoria en tiempo de ejecución sin reiniciar el emulador.

## 5.7. Telemetría de la CPU y Verificación Formal de la ALU de 64 bits

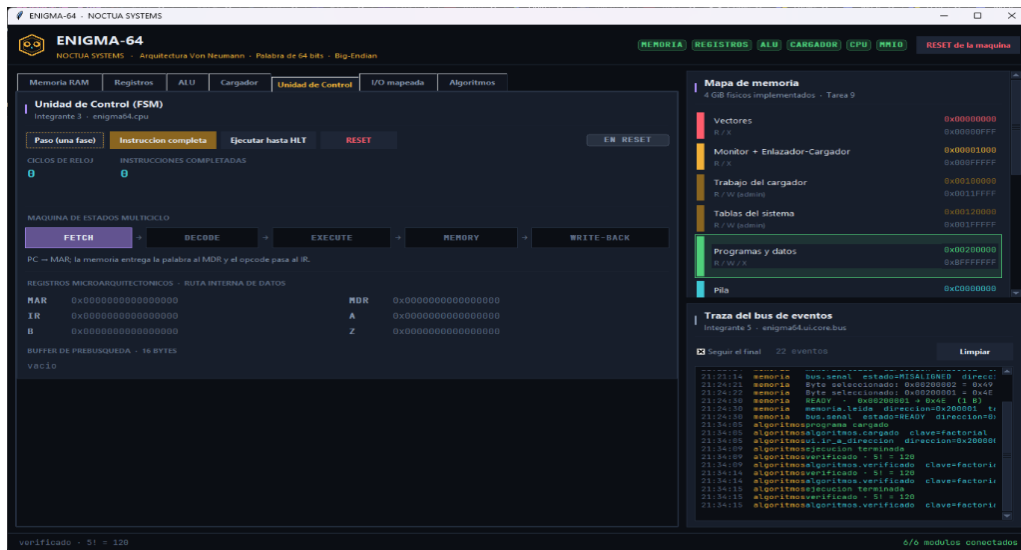


Figura 8: Panel de control de la CPU, estado de la FSM multiciclo y telemetría de micro-registros.

La Figura 8 ilustra el estado de la Unidad de Control durante la ejecución paso a paso: registros de desacoplamiento MAR, MDR, IR y los latches de operandos de la ALU, junto con el indicador visual del estado actual de la FSM.

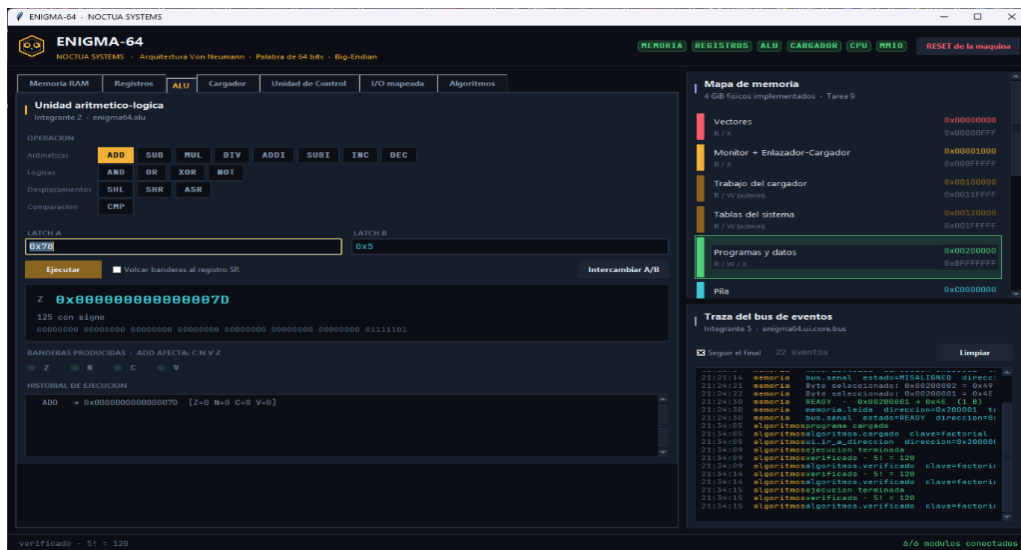


Figura 9: Panel de verificación de la ALU de 64 bits con cómputo de operandos y banderas de estado.

La Figura 9 presenta la validación interactiva de la ALU de 64 bits, evidenciando el cómputo exacto de microoperaciones aritméticas y lógicas en complemento a dos junto con la actualización coherente de las banderas Z, N, C y V.

## 5.8. Métricas y Alcance de la Suite de Pruebas Automatizadas

La verificación formal y estabilidad de Enigma-64 se sustenta en una rigurosa suite de **260 pruebas unitarias automatizadas**, las cuales alcanzan una tasa de aprobación del **100%** ejecutándose en un tiempo consolidado inferior a 0.7 segundos. El alcance funcional de la batería comprende: (a) pruebas unitarias de paginación dispersa en RAM, arbitraje de buses y detección inmediata de excepciones por desalineación natural; (b) verificación del cableado físico a tierra del registro R0 (resistencia a escrituras y lectura fija en cero) y persistencia íntegra del banco de 64 bits; (c) computación exhaustiva de operaciones en la ALU con validación de banderas aritméticas (Zero, Negative, Carry y Overflow en complemento a dos); (d) análisis sintáctico de binarios .e64 y desempaqueado estructurado en el cargador; (e) transiciones deterministas de microarquitectura en la FSM multiciclo (5 fases) y desacoplamiento de lecturas en el *PrefetchBuffer*; (f) interceptación MMIO y simulación de la matriz CRT del controlador de pantalla en 0xFF001000; (g) verificación numérica exacta de los tres programas de prueba (Factorial, Euclides y Fibonacci) comparando resultados en memoria física contra modelos formales; y (h) pruebas estáticas de aislamiento de capas mediante análisis del Árbol de Sintaxis Abstracta (AST), auditando que ningún módulo de hardware dependa de interfaces de usuario ni existan acoplamientos circulares entre paneles.

| Archivo de Suite de Pruebas     | Componente Evaluado                              | Pruebas    | Tasa de Éxito      |
|---------------------------------|--------------------------------------------------|------------|--------------------|
| tests/test_memoria.py           | RAM física, paginación dispersa y alineación     | 22         | 100% OK            |
| tests/test_registros_alu.py     | Banco de registros de 64 bits y operaciones ALU  | 38         | 100% OK            |
| tests/test_cargador.py          | Cargador .e64 y manipulación de bits             | 21         | 100% OK            |
| tests/test_cpu.py               | FSM multiciclo, prefetch buffer y conjunto ISA   | 16         | 100% OK            |
| tests/test_perifericos.py       | Dispatcher MMIO y Controlador de Pantalla CRT    | 16         | 100% OK            |
| tests/test_algoritmos.py        | Programas Factorial, Euclides y Fibonacci        | 16         | 100% OK            |
| tests/test_ui_visor_ram_mmio.py | Lógica de banco físico y edición en vivo de bits | 8          | 100% OK            |
| tests/test_ui_aislamiento.py    | Aislamiento estricto de capas mediante AST       | 29         | 100% OK            |
| tests/test_ui_modulos_nuevos.py | Contratos de integración y fábrica de servicios  | 42         | 100% OK            |
| tests/test_ui_nucleo.py         | Formatos numéricos, endianness y bus de eventos  | 52         | 100% OK            |
| <b>TOTAL INTEGRADO</b>          | <b>Emulador completo Enigma-64</b>               | <b>260</b> | <b>100% PASSED</b> |

Tabla 3: Distribución y tasa de aprobación de la batería de pruebas automatizadas.

## REFERENCIAS BIBLIOGRÁFICAS

- [1] Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
- [2] Tanenbaum, A. S., & Austin, T. (2013). *Structured Computer Organization* (6th ed.). Pearson.
- [3] Stallings, W. (2016). *Computer Organization and Architecture: Designing for Performance* (10th ed.). Pearson.
- [4] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [5] Documentación Técnica de Asignatura. (2026). *Tarea 9: Diseño de un computador propio inspirado en el computador del Prof. Peña*. Universidad Nacional de Colombia.